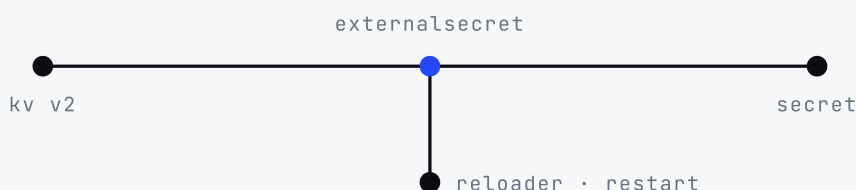


KUBERNETES SECRETS WITH ESO

Fetching static secrets from the KV engine into the cluster, rotating them, pushing them back – and why a rotated secret is not yet a rotated application.



Kubernetes Secrets with ESO and OpenBao

Fetching static secrets from the KV engine into the cluster, rotating them, pushing them back – and why a rotated secret is not yet a rotated application

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI
- A word about the values in this guide
- Who this is for
- The building blocks
- The architecture in one picture

Part I – KV v2 in OpenBao

- The mount
- Creating a secret
- Who may read: the policy

Part II – ExternalSecret: field by field

- SecretStore
- The first ExternalSecret
- Error 1: permission denied
- Switching the Deployment
- Aside: the TTL trap from N° 2, live

Part III – Rotation

- Error 2: kv put replaces, it does not add
- Error 3: the Secret is new, the pod is not
- Error 4: the application is new, the password is not
- Reloader: restart on Secret change

Part IV – The other ways of reading

- `dataFrom.extract`: the whole secret
- `dataFrom.find`: many secrets by pattern
- `template`: files and composite values
- Lifecycle: `creationPolicy` and `deletionPolicy`

Part V – PushSecret: the other direction

- Where it belongs in operation

Part VI – What went wrong, and why

Part VII – Operation

- The policy as it looks in the end
- Versions as a safety net
- Choosing `refreshInterval`
- What belongs in `OpenTofu`
- What is still open

Appendix A – All commands

Appendix B – Glossary

What this is about

Most secrets are not dynamic. A third-party API key, an admin password, a webhook token – values somebody creates once and that stay valid until somebody changes them. They still do not belong in a manifest, in Helm values or in a CI variable. They belong somewhere that knows versions, logs access and knows who may read: OpenBao's KV engine.

This guide shows how the **External Secrets Operator** (ESO) brings such values into the cluster and lays them down as a perfectly ordinary kind: Secret – field by field, as a whole secret, by pattern across many secrets, or as a rendered file. How to rotate, and everything that does *not* happen when you do. And how to write in the other direction, when a secret is born in the cluster and should be kept safe in OpenBao.

It is the fourth part of a series. N° 1 builds the OpenBao in the cluster, N° 2 lets an application draw dynamic database credentials from it, N° 3 describes the OpenBao on a VM. N° 2 skipped the static case and left the Miniflux admin password as an open item in the manifest – this is where it gets closed.

Everything was actually carried out on the cluster. The four errors that came up are the core of the guide.

Why by hand, and why without AI

ExternalSecret is an object with twenty fields, and an AI generates one in three seconds. What it does not generate is the understanding of why ESO turns key: demo/miniflux into the path secret/data/demo/miniflux, why the policy for find needs something different from extract, why kv put deletes a field you did not mention, and why a pod still has the old password after the rotation.

The yardstick: whoever has worked through this guide can say, for every field of an ExternalSecret, which OpenBao path ESO derives from it and which capability the policy needs for that. And they can explain the three places where a rotation can fail although OpenBao has had the new value for a long time.

A word about the values in this guide

Everything comes from a development environment. The passwords (admin123, admin456) are printed on purpose so you recognise them in the flow. Internal addresses are cluster DNS names that mean nothing outside the cluster.

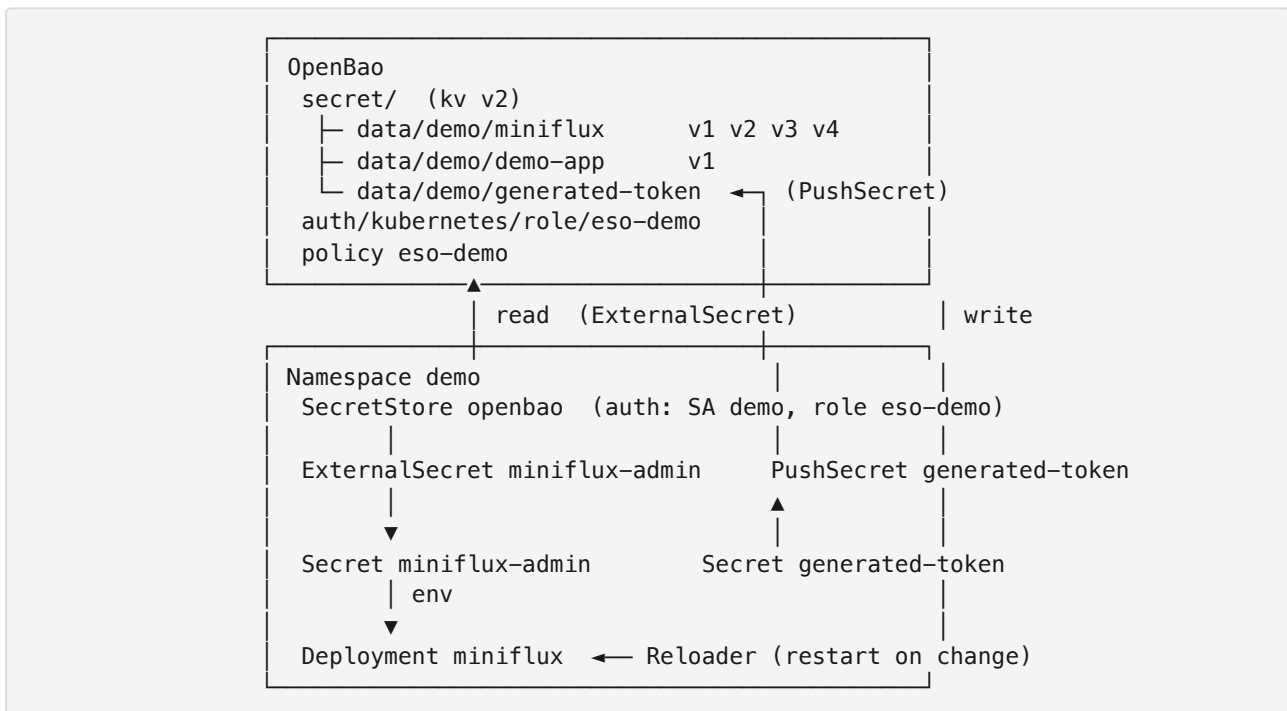
Who this is for

Readers who know N° 1 and N° 2 or bring the basics: Kubernetes Secrets, ServiceAccounts, Helm; in OpenBao the Kubernetes auth method and the concept of policies. If you want to learn KV v2, versions and policies from the ground up, that is in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

The building blocks

Component	Version	Role
OpenBao in the cluster (N° 1)	2.6.2	KV v2 engine at secret/, Kubernetes auth with role eso- demo
External Secrets Operator	2.10.0	SecretStore, ExternalSecret, PushSecret
Stakater Reloader	chart 2.2.17	restarts pods when a Secret changes
Miniflux (N° 2)	2.3.3	the application whose admin password moves here

The architecture in one picture



Three things to take from the picture:

- 1. The SecretStore is the connection, the ExternalSecret is the order.** The store says *where* and *as whom*; the ExternalSecret says *what* and *in which shape*. Many ExternalSecrets share one store.
- 2. Every way of reading needs its own capability.** `read` on `data/...` for single secrets, `list` on `metadata/...` for patterns, `create/update` on `data/...` and `metadata/...` for push. The policy grows with every feature.
- 3. ESO ends at the Kubernetes Secret.** What the pod does with it – and whether it notices changes – is not ESO's problem. That is what Reloader is for.

Part I – KV v2 in OpenBao

The mount

```
/ $ bao read sys/mounts/secret -format=json | jq .data.options
{ "version": "2" }
```

KV **v2** is versioned: every put creates a new version, old ones stay readable until `max_versions` pushes them out or somebody deletes them. For rotation that is essential – you can look up the previous value when the new one does not work.

The price of v2 is a quirk that catches every newcomer once: the API paths are not `secret/demo/miniflux`, but `secret/**data**/demo/miniflux` for the content and `secret/**metadata**/demo/miniflux` for versions and timestamps. The CLI (`bao kv get secret/demo/miniflux`) hides that; policies and ESO see the real paths.

Creating a secret

```
/ $ bao kv put secret/demo/miniflux admin_username=admin admin_password=admin123
created_time      2026-09-17T10:31:03.905952914Z
version          1
```

A KV secret is a map of fields. `admin_username` and `admin_password` are two fields of one secret, not two secrets. That determines how ESO fetches them later.

```
/ $ bao kv list secret/demo
Keys
----
demo-app
miniflux
```

Who may read: the policy

ESO logs in with the ServiceAccount `demo` via the Kubernetes auth role `eso-demo` and receives the policy of the same name (N° 1, Part III). Before this guide it looked like this:

```
path "secret/data/demo/demo-app"      { capabilities = ["read"] }
path "secret/metadata/demo/demo-app"  { capabilities = ["read"] }
path "database/creds/demo-app"        { capabilities = ["read"] }
path "database/creds/miniflux"        { capabilities = ["read"] }
```

`secret/demo/miniflux` is not in it. That is deliberate – the first error is meant to happen, so that you recognise it when it happens for real later.

Part II – ExternalSecret: field by field

SecretStore

Exists since N° 2, here in full:

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: openbao
  namespace: demo
spec:
  provider:
    vault:
      server: http://openbao.openbao.svc:8200
      path: secret          # the mount
      version: v2           # -> ESO inserts data/ and metadata/ itself
      auth:
        kubernetes:
          mountPath: kubernetes
          role: eso-demo
          serviceAccountRef:
            name: demo
```

A SecretStore applies to one namespace. A ClusterSecretStore applies cluster-wide and is referenced via `secretStoreRef.kind: ClusterSecretStore` – handy when many namespaces use the same OpenBao, but then every request runs under *one* identity, and the policy can no longer distinguish by namespace. One store per namespace with its own auth role is the cleaner choice as soon as there is more than one tenant.

The first ExternalSecret

Two fields, two keys in the Kubernetes Secret:

```

apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: miniflux-admin
  namespace: demo
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: SecretStore
    name: openbao
  target:
    name: miniflux-admin
    creationPolicy: Owner
  data:
    - secretKey: ADMIN_USERNAME
      remoteRef:
        key: demo/miniflux
        property: admin_username
    - secretKey: ADMIN_PASSWORD
      remoteRef:
        key: demo/miniflux
        property: admin_password

```

remoteRef.key is the path **without** the mount and **without** data/ – ESO adds both from the store. property selects one field. secretKey names the key in the target Secret; here exactly as Miniflux expects the variables.

Error 1: permission denied

```

$ kubectl apply -f k8s/miniflux-admin-external-secret.yaml
$ kubectl get externalsecret -n demo miniflux-admin
NAME                STATUS              READY   LAST SYNC
miniflux-admin      SecretSyncedError   False

```

The status.conditions[].message only says could not get secret data from provider. The cause is in the operator's log:

```

$ kubectl logs -n external-secrets deploy/external-secrets | grep miniflux-admin
error processing spec.data[0] (key: demo/miniflux), err: cannot read secret
data from Vault: Error making API request.

URL: GET http://openbao.openbao.svc:8200/v1/secret/data/demo/miniflux
Code: 403. Errors:
  * permission denied

```

Two things are instructive here. First the path: secret/data/demo/miniflux – from key: demo/miniflux plus path: secret plus version: v2. Exactly this path has to be in the policy. Second, where the message lives: the ExternalSecret only shows *that* it fails; *why* is with the operator.

Extend the policy by one line:

```

path "secret/data/demo/miniflux"    { capabilities = ["read"] }

```


And instead of waiting an hour for the next sync, force it – any change to an annotation on the ExternalSecret triggers a reconcile:

```
$ kubectl annotate externalsecret -n demo miniflux-admin force-sync="$(date +%s)" --overwrite
$ kubectl get externalsecret -n demo miniflux-admin
NAME              STATUS      READY   LAST SYNC
miniflux-admin    SecretSynced  True    5s

$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.type} {.data}'
Opaque {"ADMIN_PASSWORD":"...","ADMIN_USERNAME":"..."}

$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.metadata.ownerReferences[0].kind}'
ExternalSecret
```

The Secret is owned by the ExternalSecret (creationPolicy: Owner). Delete the ExternalSecret and garbage collection takes the Secret with it. More on that in Part IV.

Switching the Deployment

In N° 2 the admin password sat in plain text in the manifest. Now:

```
- name: ADMIN_USERNAME
  valueFrom:
    secretKeyRef:
      name: miniflux-admin
      key: ADMIN_USERNAME
- name: ADMIN_PASSWORD
  valueFrom:
    secretKeyRef:
      name: miniflux-admin
      key: ADMIN_PASSWORD
```

```
$ kubectl apply -f k8s/miniflux-deployment.yaml
$ kubectl rollout status -n demo deploy/miniflux
deployment "miniflux" successfully rolled out
$ kubectl logs -n demo deploy/miniflux --tail=3
level=INFO msg="Running database migrations" current_version=132 latest_version=132
level=INFO msg="Skipping admin user creation because it already exists" username=admin
level=INFO msg="Starting HTTP server" listen_address=0.0.0.0:8080
```

The second line is inconspicuous and becomes important in Part III: Miniflux only creates the admin if it does not exist yet. ADMIN_PASSWORD is **never read again** after that.

Aside: the TTL trap from N° 2, live

During the rollout the old pod kept running for a moment, and its log showed:

```
level=ERROR msg="Unable to fetch jobs from database" error="... pq: password
authentication failed for user \"v-kubernetes-miniflux-SkwRAKNo7i4xy0uFFD63-1789634099\""
```

The dynamic database user the pod had started with two hours earlier no longer existed – lease expired, DROP ROLE executed. The pod had never seen the new DATABASE_URL, because environment variables are only read at startup. N° 2 had named this as an open risk; here it happened. The solution comes in Part III and applies to both secrets.

Part III – Rotation

Error 2: kv put replaces, it does not add

The obvious way to change a password:

```
/ $ bao kv put secret/demo/miniflux admin_password=admin456
version          2
/ $ bao kv get -format=json secret/demo/miniflux | jq '.data.data | keys'
[ "admin_password" ]
```

admin_username is gone. kv put writes a **new version with exactly the fields given**. Version 1 still has both, but ESO reads the current one:

```
$ kubectl get externalsecret -n demo miniflux-admin
NAME              STATUS              READY   LAST SYNC
miniflux-admin    SecretSyncedError   False   73s

$ kubectl logs -n external-secrets deploy/external-secrets | grep miniflux-admin
error processing spec.data[0] (key: demo/miniflux), err: cannot find secret
data for key: "admin_username"
```

What ESO does with the Kubernetes Secret in this state is the most important property for operations:

```
$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.data.ADMIN_PASSWORD}' | base64
-d
admin123
```

Nothing. The Secret keeps the last successfully synchronised state. An error in OpenBao or in the policy does not take a running application's credentials away – it only prevents it from getting new ones. That is correct, and you have to know it, because a False under READY therefore does not mean something *is* broken – only that it is no longer being *updated*.

The right command is kv patch:

```
/ $ bao kv patch secret/demo/miniflux admin_username=admin
version          3
/ $ bao kv get -format=json secret/demo/miniflux | jq '.data.data | keys'
[ "admin_password", "admin_username" ]
```

```
$ kubectl annotate externalsecret -n demo miniflux-admin force-sync="$(date +%s)" --overwrite
$ kubectl get externalsecret -n demo miniflux-admin
NAME              STATUS              READY   LAST SYNC
miniflux-admin    SecretSynced        True    5s
$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.data.ADMIN_PASSWORD}' | base64
-d
admin456
```

Error 3: the Secret is new, the pod is not

```
$ kubectl get pod -n demo -l app=miniflux -o custom-columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
NAME                                CREATED
miniflux-86c6489dc4-6p2j5         2026-09-17T10:31:57Z

$ kubectl exec -n demo deploy/miniflux -- sh -c 'echo $ADMIN_PASSWORD'
admin123
```

The Kubernetes Secret says admin456, the container says admin123. A container reads env at startup, and nothing restarted it. Kubernetes updates mounted Secrets inside a running container (with a delay), but environment variables never.

Error 4: the application is new, the password is not

Suppose the pod had restarted and had admin456 in its environment. Would the login work?

```
$ kubectl run -n demo --rm -i --restart=Never --image=curlimages/curl:8.11.1 t -- \
  sh -c 'for pw in admin456 admin123; do
    printf "admin/%s -> " $pw
    curl -s -o /dev/null -w "%{http_code}\n" -u admin:$pw http://<pod-ip>:8080/v
1/me
    done'
admin/admin456 -> 401
admin/admin123 -> 200
```

No. Miniflux created the admin with admin123 in its database at the very first start and has not read ADMIN_PASSWORD since (“Skipping admin user creation because it already exists”). The password in OpenBao, in the Kubernetes Secret and in the container is admin456; the password that counts lives in the Miniflux database and is admin123.

Rotation in the secrets store is not rotation in the application. For every value you have to know whether the application picks it up on every start (database URLs, API keys for third-party services: yes) or only the first time (bootstrap passwords, initial admins: no). For Miniflux the rotation goes through `miniflux -reset-password` – or, more consistently, by not setting the admin password through the environment at all any more, only once at bootstrap.

That is why dynamic credentials (N° 2) are worth so much: there is no state in the application that can go stale.

Reloader: restart on Secret change

For error 3 there is a standard solution. Stakater Reloader watches Secrets and ConfigMaps and rolls out every Deployment that references them:

```
helm repo add stakater https://stakater.github.io/stakater-charts
helm upgrade --install reloader stakater/reloader --version 2.2.17 \
  --namespace reloader --create-namespace \
  --set reloader.watchGlobally=true
```

One annotation on the Deployment is enough:

```
metadata:
  name: miniflux
  namespace: demo
  annotations:
    reloader.stakater.com/auto: "true"
```

Then the next rotation – here back to admin123, so that Secret and application agree again:

```
/ $ bao kv patch secret/demo/miniflux admin_password=admin123
version          4
$ kubectl annotate externalsecret -n demo miniflux-admin force-sync="$(date +%s)" --overw
ite

$ kubectl logs -n reloader deploy/reloader-reloader | grep miniflux
level=info msg="Changes detected in 'miniflux-admin' of type 'SECRET' in
namespace 'demo'; updated 'miniflux' of type 'Deployment' in namespace 'demo'"

$ kubectl get pod -n demo -l app=miniflux -o custom-columns=NAME:.metadata.name,CREATED:.m
etadata.creationTimestamp
NAME                                CREATED
miniflux-7bf56c686c-lfss7          2026-09-17T10:34:17Z

$ kubectl exec -n demo deploy/miniflux -- sh -c 'echo $ADMIN_PASSWORD'
admin123
```

How Reloader does it is visible in the Deployment:

```
$ kubectl get deploy -n demo miniflux -o jsonpath='{.spec.template.spec.containers[0].env
[*].name}'
... STAKATER_MINIFLUX_ADMIN_SECRET
```

It appends an environment variable holding the hash of the Secret to the container. If the hash changes, the pod template changes, and Kubernetes rolls out – an ordinary rolling update, with everything that entails (readiness, maxUnavailable).

A side effect that closes the TTL trap from N° 2: auto: "true" applies to **all** referenced Secrets, so also to miniflux-db. Every 30 minutes ESO fetches new database credentials, Reloader restarts Miniflux, and the pod never runs with an expired user. For an RSS reader, a restart every 30 minutes is fine. For an application where it is not, the TTL has to be longer, or the Secret mounted as a file and re-read by the application.

Part IV – The other ways of reading

data with property fetches single fields. Three more shapes, all tested on the cluster.

dataFrom.extract: the whole secret

```
spec:
  target:
    name: demo-app-extract
  dataFrom:
    - extract:
        key: demo/demo-app
```

Every field of the KV secret becomes a key in the Kubernetes Secret, without listing them:

```
$ kubectl get secret -n demo demo-app-extract -o jsonpath='{.data}'
{"password":"...","username":"..."}
```

Same policy as for data – read on secret/data/demo/demo-app.

dataFrom.find: many secrets by pattern

```
spec:
  target:
    name: demo-find
  dataFrom:
    - find:
        path: demo
        name:
          regexp: ".*"
```

On the first attempt:

```
error processing spec.dataFrom[0].find, err: error getting all secrets:
cannot read secret data from Vault: Error making API request.

URL: GET http://openbao.openbao.svc:8200/v1/secret/metadata/demo?list=true
Code: 403.
```

find first has to know *which* secrets exist, and for that it lists the metadata path. That is a different capability from read:

```
path "secret/metadata/demo" { capabilities = ["list"] }
```

And one to grant deliberately: whoever has `list` sees the names of all secrets under the prefix – including those they may not read. Afterwards:

```
$ kubectl get secret -n demo demo-find -o jsonpath='{.data}'
{"demo_demo-app":"...", "demo_miniflux":"..."}

$ kubectl get secret -n demo demo-find -o jsonpath='{.data.demo_miniflux}' | base64 -d
{"admin_password":"...", "admin_username":"..."}
```

One key per secret found (path with `_` instead of `/`), the value is the whole secret as JSON. Usable for applications that parse their own configuration; unsuitable for env variables.

template: files and composite values

```
spec:
  target:
    name: demo-app-template
    template:
      engineVersion: v2
      data:
        app.env: |
          APP_USERNAME={{ .username }}
          APP_PASSWORD={{ .password }}
        basic-auth: "{{ .username }}:{{ .password | b64enc }}"
      dataFrom:
        - extract:
            key: demo/demo-app
```

```
$ kubectl get secret -n demo demo-app-template -o jsonpath='{.data.app\.env}' | base64 -d
APP_USERNAME=...
APP_PASSWORD=...
```

The fields from `data/dataFrom` are available in the template as `.name`, plus the Sprig functions (`b64enc`, `upper`, `toJson`, ...). That turns two fields into a `.env` file, a connection URL (Nº 2 uses exactly this for `DATABASE_URL`) or an `htpasswd` line.

Lifecycle: creationPolicy and deletionPolicy

Field	Values	Meaning
creationPolicy	Owner (default), Orphan, Merge, None	Owner: ESO creates and owns the Secret. Merge: ESO writes only its keys into an existing Secret. None: ESO creates nothing
deletionPolicy	Retain (default), Delete, Merge	What happens when the value disappears in OpenBao: Retain keeps the Secret, Delete removes it

What happens when the **ExternalSecret** is deleted depends on `creationPolicy`, not on `deletionPolicy`:

```
$ kubectl delete externalsecret -n demo demo-app-extract
$ kubectl get secret -n demo demo-app-extract
Error from server (NotFound): secrets "demo-app-extract" not found
```

Owner → OwnerReference → garbage collection. Whoever wants to keep the Secret after deleting the ExternalSecret uses Orphan.

Part V – PushSecret: the other direction

Some secrets are born in the cluster: a token an operator generates, a certificate from cert-manager, a kubeconfig from a provisioner. Writing them into OpenBao has two advantages: they are covered by the Raft snapshots, and every read is in the audit log.

```
apiVersion: external-secrets.io/v1alpha1
kind: PushSecret
metadata:
  name: generated-token
  namespace: demo
spec:
  refreshInterval: 1h
  secretStoreRefs:
    - kind: SecretStore
      name: openbao
  selector:
    secret:
      name: generated-token
  data:
    - match:
        secretKey: token
        remoteRef:
          remoteKey: demo/generated-token
          property: token
```

First attempt, with a policy that only allows create/update on secret/data/demo/generated-token:

```
URL: GET http://openbao.openbao.svc:8200/v1/secret/data/demo/generated-token
Code: 403.
```

PushSecret **reads first**, to compare whether anything has changed. So add read. Second attempt:

```
URL: PUT http://openbao.openbao.svc:8200/v1/secret/metadata/demo/generated-token
Code: 403.
```

After writing the data, ESO writes custom_metadata – a marker that this secret is managed by ESO. That is a write to the **metadata** path. The complete policy for one push path:

```
path "secret/data/demo/generated-token" { capabilities = ["create", "read", "update"]
}
path "secret/metadata/demo/generated-token" { capabilities = ["create", "read", "update"]
}
```

```
$ kubectl get pushsecret -n demo generated-token -o jsonpath='{.status.conditions[0].message}'
PushSecret synced successfully

/ $ bao kv metadata get secret/demo/generated-token | grep -A1 custom_metadata
custom_metadata      map[managed-by:external-secrets]
```

Three paths and three errors for a single push – the policy for PushSecret is considerably broader than for ExternalSecret. Restrict it to exactly the paths that are pushed, and never make secret/data/* writable just because a push does not work at first try.

Where it belongs in operation

ESO recommends not pointing PushSecret and ExternalSecret at the same path – two controllers managing the same truth are an oscillator. Push for what the cluster generates; External for what OpenBao manages.

Part VI – What went wrong, and why

Error	Message	Cause	Fix
1	GET .../secret/data/demo/miniflux 403	policy does not know the path	read on secret/data/<path>
2	cannot find secret data for key: "admin_username"	kv put deleted the field	kv patch; version 1 still has the value
3	Secret new, container has old value	env is read only at startup	Reloader, or mount the Secret as a file
4	login with new password → 401	application reads the value only at bootstrap	rotate in the application, or dynamic credentials
find	GET .../secret/metadata/demo?list=true 403	find lists metadata	list on secret/metadata/<prefix>
Push	GET .../data/... 403, then PUT .../metadata/... 403	push reads, writes data, writes metadata	create,read,update on both paths

And the pattern behind it: **READY False does not mean broken**. ESO leaves the Kubernetes Secret untouched as long as the sync fails. The application keeps running with the last good state. The cause is not in the ExternalSecret but in the operator's log:

```
kubectl logs -n external-secrets deploy/external-secrets | grep <name>
```

Part VII – Operation

The policy as it looks in the end

```
path "secret/data/demo/demo-app"      { capabilities = ["read"] }
path "secret/metadata/demo/demo-app"  { capabilities = ["read"] }
path "secret/data/demo/miniflux"      { capabilities = ["read"] }
path "secret/metadata/demo"           { capabilities = ["list"] }
path "secret/data/demo/generated-token" { capabilities = ["create", "read", "update"]
}
path "secret/metadata/demo/generated-token" { capabilities = ["create", "read", "update"]
}
path "database/creds/demo-app"         { capabilities = ["read"] }
path "database/creds/miniflux"        { capabilities = ["read"] }
```

Eight lines, each added because of a concrete error. That is the right way to build a policy – not `secret/*` with all capabilities, but path by path, with the error as evidence.

Versions as a safety net

```
/ $ bao kv get -version=1 secret/demo/miniflux
```

After a botched `kv put`, the old state is one version back. `bao kv rollback -version=1 secret/demo/miniflux` makes it the current version (as new version 5; the history stays). ESO fetches it on the next sync.

Choosing `refreshInterval`

Static secrets change rarely; 1h is a good default. Shorter brings nothing but load on OpenBao and entries in the audit log. For the moment right after a rotation there is the annotation. `0` switches the periodic sync off – ESO then syncs only on changes to the `ExternalSecret`.

What belongs in OpenTofu

The policy, the Kubernetes auth role, the KV mount. **Not** the secret values – those originate in the password manager and are entered with `bao kv put`, or they are written from the cluster via `PushSecret`. A `vault_kv_secret_v2` with the password in HCL would be exactly the mistake ESO is meant to avoid.

What is still open

- The Miniflux admin password is set in the environment on every start but only used on the first. Cleaner: `CREATE_ADMIN` only for the bootstrap, then removed from the manifest.

- The ESO operator itself has cluster-wide rights on Secrets. Whoever compromises it reads them all. That is the price of the pattern, and it is the reason the OpenBao policy per namespace should be as tight as possible.

Appendix A – All commands

```
# — KV —————
bao kv put    secret/demo/miniflux admin_username=admin admin_password=admin123
bao kv patch secret/demo/miniflux admin_password=admin456      # change one field
bao kv get -version=1 secret/demo/miniflux                    # read an old version
bao kv rollback -version=1 secret/demo/miniflux              # go back
bao kv metadata get secret/demo/miniflux

# — Policy (root or admin) —————
bao policy write eso-demo - < k8s/eso-demo-policy.hcl

# — ESO —————
kubectl apply -f k8s/secretstore.yaml
kubectl apply -f k8s/miniflux-admin-external-secret.yaml
kubectl apply -f k8s/demo-app-extract.yaml -f k8s/demo-find.yaml -f k8s/demo-app-template.
yaml
kubectl apply -f k8s/push-secret.yaml
kubectl get externalsecret,pushsecret -n demo
kubectl annotate externalsecret -n demo <name> force-sync="$(date +%s)" --overwrite
kubectl logs -n external-secrets deploy/external-secrets | grep <name>

# — Reloader —————
helm upgrade --install reloader stakater/reloader --version 2.2.17 \
  -n reloader --create-namespace --set reloader.watchGlobally=true
kubectl annotate deploy -n demo miniflux reloader.stakater.com/auto="true"
kubectl logs -n reloader deploy/reloader-reloader | grep <secret>
```

Appendix B – Glossary

KV v2 – Versioned key-value engine. Paths `data/` (content) and `metadata/` (versions, timestamps, `custom_metadata`).

SecretStore / ClusterSecretStore – ESO object describing provider, address and auth. Namespace-bound or cluster-wide.

ExternalSecret – Order to ESO to write values from the store into a Kubernetes Secret. `data` (fields), `dataFrom.extract` (whole secret), `dataFrom.find` (pattern).

PushSecret – The other direction: Kubernetes Secret → store.

remoteRef.key – Path in the store without mount and without `data/`; ESO adds both.

property – One field inside a KV secret.

template – Go template with Sprig functions that builds the keys of the target Secret from the fetched fields.

creationPolicy – Who owns the target Secret (Owner, Orphan, Merge, None).

deletionPolicy – What happens when the value disappears from the store.

refreshInterval – How often ESO queries the store. An annotation change forces a sync immediately.

Reloader – Controller that rolls out Deployments when referenced Secrets or ConfigMaps change; appends a hash as an env variable to do so.

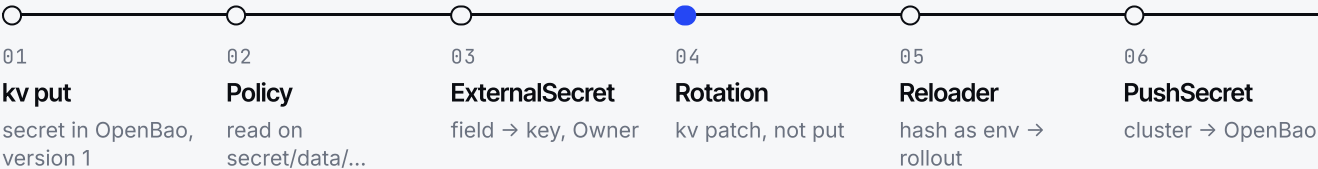
kv put vs. kv patch – `put` writes a new version with exactly the fields given; `patch` changes only the fields given.

The Secret says admin456. The application accepts admin123. Both are right.

This guide brings static secrets from OpenBao into the cluster with the External Secrets Operator – field by field, as a whole secret, by pattern, as a rendered file – and writes them back with PushSecret. Every variant tested on the cluster, with the policy line it needs.

And four errors that lie in wait for every rotation: the policy that does not know the path; kv put deleting a field; the pod that never reads the new Secret; the application that only takes the password on its first start. Reloader solves one of them.

```
-- The difference that deleted a field
bao kv put    secret/demo/miniflux admin_password=... # replaces everything
bao kv patch  secret/demo/miniflux admin_password=... # changes one
```



openbao	2.6.2 · kv v2 · secret/
external-secrets	2.10.0 · v1 · PushSecret v1alpha1
reloader	chart 2.2.17 · watchGlobally
miniflux	2.3.3
policy	8 lines · 6 errors

24 pages · 7 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.